claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a823b2c089c52888e.jsonl`
- SHA-256: `48d3ddc113ae5f4dfe1ab4703168c43622f32bd08305b6f20717206b3c91071d`
- Source modified: `2026-06-22T09:48:50+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:45:35.971Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:
{
 "title": "CRITICAL: UPTIME calculation can silently underflow to zero if BOOT_EPOCH fails to set",
 "file": "deploy/gcp/nightly_regression/run_on_vm.sh",
 "line": "15, 80",
 "severity": "high",
 "detail": "Line 15: BOOT_EPOCH=$(date +%s) ? if this fails (theoretically low probability on a fresh VM but non-zero, e.g., timezone database corrupted), BOOT_EPOCH is unset. Line 80: UPTIME=$(($(date +%s) - BOOT_EPOCH)) ? with unset BOOT_EPOCH, bash arithmetic treats it as 0, causing UPTIME to be the full wall time, which is INCORRECT for cost billing. The Python script receives a falsely inflated uptime (the entire pipeline runs at once rather than measuring boot overhead separately). This could cause billing cost estimates to be significantly overstated or understated.",
 "fix": "Validate BOOT_EPOCH after setting: BOOT_EPOCH=$(date +%s) || { echo \"FATAL: cannot read system time\"; exit 1; }. Also add a sanity check before passing to Python: [\"$UPTIME\" -lt 5] && echo \"WARN: UPTIME suspiciously low ($UPTIME s)\""
}

2. assistant (2026-06-22T09:45:38.378Z)

I'll systematically analyze these files to verify the claimed defect about UPTIME calculation underflow. Let me start by reading all the relevant files.

3. user (2026-06-22T09:45:39.372Z)

1	#!/usr/bin/env bash
2	# Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then DELETES the VM
3	# (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a startup-script, not

```
4       # an interactive session). Reads its config from instance metadata (set by launch.sh).
5       #
6       # Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
7       #        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8       #        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
9       #
10      # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11      #   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
```

```
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

4. user (2026-06-22T09:45:40.543Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
```

(``deploy/gcp/nightly_regression/``), emails the
8   owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10   What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11   ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12   Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13   **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14   ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15   ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16   ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18   Extensible pattern (add a video = one manifest row + re-freeze):
19   ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20   ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22   Usage
23   -----
24   Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25   python scripts/nightly_reelcount.py --update-baseline
26
27   Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28   python scripts/nightly_reelcount.py --email
29
30   Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31   (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32   """
33   from __future__ import annotations
34
35   import argparse
36   import datetime as _dt
37   import json
38   import os
39   import sys
40   from dataclasses import dataclass, field
41   from typing import Any, Dict, List, Optional, Tuple
42
43   REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44   if REPO_ROOT not in sys.path:
45   sys.path.insert(0, REPO_ROOT)
46
47   # billing is pure (typing only) ? safe to import at module top.
48   from backend import billing # noqa: E402
49
50   DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51   DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52   DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54   #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55   QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56   DEFAULT_QUALITY_TOL = 0.02   # quality is float/labeled ? a looser tripwire than the

```
exact reel-count
    DEFAULT_FX_INR_PER_USD = 83.0
    #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
    DEFAULT_MACHINE_USD_PER_HR = 0.35


    @dataclass(frozen=True)
    class Tolerances:
        quality_tol: float = DEFAULT_QUALITY_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"quality_tol": self.quality_tol}


    # ---------------------------------------------------------------------- #
    # Pure cost math (wraps backend.billing ? no IO).
    # ---------------------------------------------------------------------- #
    def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
        """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).

        ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
        otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
        rate_per_sec = float(machine_usd_per_hr) / 3600.0
        usage = {billing.WALL_SECONDS: float(billed_seconds)}
        rates = {billing.WALL_SECONDS: rate_per_sec}
        usd, breakdown = billing.compute_cost_usd(usage, rates)
        return {
            "billed_seconds": round(float(billed_seconds), 1),
            "gpu_seconds": round(float(gpu_seconds), 1),
            "machine_usd_per_hr": float(machine_usd_per_hr),
            "usd": usd,
            "inr": billing.to_inr(usd, fx_inr_per_usd),
            "breakdown": breakdown,
        }


    # ---------------------------------------------------------------------- #
    # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
    # ---------------------------------------------------------------------- #
    def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                          cost: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable snapshot from per-video run results.

        Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
        A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
        per_video: Dict[str, Any] = {}
        for r in run_results:
            per_video[r["name"]] = {
                "reel_count": r.get("reel_count"),
                "ok": bool(r.get("ok", False)),
                "error": r.get("error"),
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}


    @dataclass
```

```
    class Finding:
        video: str
        metric: str                # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
        kind: str                  # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        detail: str = ""

        def message(self) -> str:
            if self.kind == "stale":
                return f"[STALE] {self.metric}: {self.detail}"
            if self.metric == "error":
                return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
            b = "?" if self.baseline is None else f"{self.baseline:g}"
            c = "?" if self.current is None else f"{self.current:g}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


    @dataclass
    class NightlyResult:
        findings: List[Finding] = field(default_factory=list)
        added: List[str] = field(default_factory=list)
        removed: List[str] = field(default_factory=list)

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]

        @property
        def stale(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "stale"]

        def overall_status(self) -> str:
            if self.regressions:
                return "REGRESSION"
            if self.stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.regressions:
                return 1
            if self.stale:
                return 4
            return 0


    def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
        """Diff current run vs frozen baseline.

        * Segmenter changed ? STALE (numbers not comparable; re-freeze).
        * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
        * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
          quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
        """
        res = NightlyResult()
```

```
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count
produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
```

```
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ------------------------------------------------------------------------ #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
        # CSV: header start,end (or first two numeric columns)
        import csv
        out: List[Tuple[float, float]] = []
        with open(local, encoding="utf-8") as fh:
            for row in csv.reader(fh):
                try:
                    out.append((float(row[0]), float(row[1])))
                except (ValueError, IndexError):
                    continue
```

```
        return out


    def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
        """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
        codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
        optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
        quality scoring gracefully (skipped) without ever miscounting reels."""
        out: List[Tuple[float, float]] = []
        for s in segs:
            start = s.get("start_time", s.get("start"))
            end = s.get("end_time", s.get("end"))
            if start is not None and end is not None:
                out.append((float(start), float(end)))
        return out


    def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                      rerun_on_mismatch: bool = True,
                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
        """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.

        The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
        database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
        reproduce, a real change will."""
        import random
        import time

        from backend.eval import rally_seg_eval
        from backend.infrastructure.database import Database
        from backend.pipeline.segmenters import segmenter_registry
        from backend.results import Failure
        from backend.storage import fetch
        from backend.utils.hashing import compute_video_id

        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
```

```
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
        parts = dotted.split(".")
```

```
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
                            "defaults to the in-process pipeline wall time (a lower bound)")
        p.add_argument("--no-rerun-guard", action="store_true",
                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
```

```
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
                log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
            return 0

        if baseline is None:
            log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
            return 3

        result = compare(baseline, snapshot, tols)
        report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

        if not args.no_report:
            os.makedirs(args.report_dir, exist_ok=True)
```

```
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

5. user (2026-06-22T09:45:40.722Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6       never mask a regression).
7
8       Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
10        ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12        ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
13                      (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                       lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
```

```
the simplest.
15
16      Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17      and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                      export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19      """
20      from __future__ import annotations
21
22      import logging
23      import os
24      from email.message import EmailMessage
25      from typing import Optional
26
27      log = logging.getLogger("nightly_email")
28
29      DEFAULT_TO = "avi.dullu@gmail.com"
30      DEFAULT_HOST = "smtp.gmail.com"
31      DEFAULT_PORT = 587
32
33
34      def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35          """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36          msg = EmailMessage()
37          msg["Subject"] = subject
38          msg["From"] = from_addr
39          msg["To"] = to_addr
40          msg.set_content(body_md)
41          return msg
42
43
44      def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45          """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46          env = env if env is not None else os.environ
47          direct = env.get("NIGHTLY_SMTP_PASS")
48          if direct:
49              return direct
50          secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51          if not secret_ref:
52              return None
53          # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54          try:
55              from google.cloud import secretmanager  # type: ignore
56              client = secretmanager.SecretManagerServiceClient()
57              return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58          except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59              pass
60          try:
61              import subprocess
62              name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63              out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                   capture_output=True, text=True, timeout=30)
65              return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66          except Exception as e:  # noqa: BLE001
67              log.warning("secret-manager resolution failed: %s", e)
68              return None
```

```
69
70
71      def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                      env: Optional[dict] = None) -> bool:
73          """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74          env = env if env is not None else os.environ
75          to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76          host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77          port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78          user = env.get("NIGHTLY_SMTP_USER") or to_addr
79          password = resolve_secret(env)
80          if not password:
81              log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82              return False
83
84          import smtplib
85          msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86          with smtplib.SMTP(host, port, timeout=60) as s:
87              s.starttls()
88              s.login(user, password)
89              s.send_message(msg)
90          log.info("emailed report to %s", to_addr)
91          return True
92
```

6. user (2026-06-22T09:45:41.803Z)

```
1       #!/usr/bin/env bash
2       # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
VM runs
3       # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
stages the repo
4       # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a
cron on any
5       # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
checkout.
6       #
7       #   bash deploy/gcp/nightly_regression/launch.sh                # L4, stages HEAD,
kicks the VM
8       #   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
9       #
10      # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
? the VM-side
11      # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
<name> --zone=<z>.
12      set -uo pipefail
13
14      PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
15      BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
16      SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
17      GPU="${RALLY_GPU:-l4}"
18      NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
19      REF="${RALLY_REF:-HEAD}"
20      WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
21      REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
22      DO_EMAIL="${RALLY_EMAIL:-true}"
23      SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
pass/versions/latest}"
24      SMTP_USER="${NIGHTLY_SMTP_USER:-}"
25      EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
26      HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
27      STARTUP="$HERE/run_on_vm.sh"
```

```
28      export CLOUDSDK_CORE_DISABLE_PROMPTS=1
29
30      ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
31      if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
32
33      # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
bucket).
34      SHA="$(git rev-parse "$REF")"
35      TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36      echo "== staging repo $SHA -> $TGZ_URI =="
37      TMP_TGZ="$(mktemp --suffix=.tgz)"
38      git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39      gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40      rm -f "$TMP_TGZ"
41
42      # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43      ERR="$(mktemp)"
44      for Z in $ZONES; do
45        echo "== creating $NAME ($GPU) in $Z =="
46        if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55          echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and
self-delete."
56          echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57          echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58          rm -f "$ERR"; exit 0
59        fi
60        if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in
$Z ? next zone)"; continue; fi
61        echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62      done
63      echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64      rm -f "$ERR"; exit 1
65
```

7. user (2026-06-22T09:45:42.280Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
```

```
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

8. assistant (2026-06-22T09:45:44.999Z)

Now let me examine the specific lines mentioned in the finding and verify the claim about UPTIME underflow.

9. user (2026-06-22T09:45:46.133Z)

```
1       #!/usr/bin/env bash
2       # Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
3       #   Cloud Scheduler (cron)  ?  Cloud Run JOB (the launcher image)  ?  launch.sh
4       #     ?  ephemeral GPU VM (regression + email + SELF-DELETE).
5       # The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU
VM is ephemeral).
6       #
7       # ? OWNER-RUN, ONCE, after a successful manual `bash
deploy/gcp/nightly_regression/launch.sh` dry-run.
8       # NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips
+ weights in
9       # gs://<bucket>, and the SMTP secret (see README "Enable").
10      #
11      #   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy
+ schedule (03:00 IST)
12      #   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
13      set -euo pipefail
14
15      PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
16      REGION="${RALLY_RUN_REGION:-asia-south1}"
17      JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
18      SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
19      CRON="${RALLY_CRON:-0 3 * * *}"          # 03:00 daily
20      TZ="${RALLY_TZ:-Asia/Kolkata}"
21      SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
22      IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-
launcher:latest}"
23      REPO_ROOT="$(git rev-parse --show-toplevel)"
24      export CLOUDSDK_CORE_DISABLE_PROMPTS=1
25
26      echo "== enable APIs (run, scheduler, build, artifact registry) =="
```

```
27      gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
28        cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"
29
30      echo "== ensure the Artifact Registry repo 'rally' exists =="
31      gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 || \
32        gcloud artifacts repositories create rally --repository-format=docker
--location="$REGION" --project="$PROJECT"
33
34      echo "== build + push the launcher image from the repo root =="
35      ( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
36          --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-
corpus}/cloudbuild" \
37          -f deploy/gcp/nightly_regression/Dockerfile.launcher . )
38
39      echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU
VM) =="
40      gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
41        --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
42        --set-env-vars="RALLY_GPU=${RALLY_GPU:-
l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
43
44      echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
45      RUN_URI="https://${REGION}-
run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
46      gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 && \
47        gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT"
--quiet || true
48      gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
49        --schedule="$CRON" --time-zone="$TZ" \
50        --uri="$RUN_URI" --http-method=POST \
51        --oauth-service-account-email="$SA"
52
53      cat <<EOF
54
55      ? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
56         IAM the launcher SA needs (grant once if missing):
57           roles/compute.instanceAdmin.v1  (create/delete the GPU VM)
58           roles/iam.serviceAccountUser    (attach $SA to the VM)
59           roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
60         Trigger now to test:  gcloud run jobs execute $JOB --region=$REGION
--project=$PROJECT
61         Remove:               gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud
run jobs delete $JOB --region=$REGION
62      EOF
63
```

10. user (2026-06-22T09:45:47.013Z)

```
1       # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2       # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3       # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4       FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
5       WORKDIR /app
6       # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7       COPY . /app
8       ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9
```

11. assistant (2026-06-22T09:45:49.211Z)

Now let me carefully analyze the specific lines mentioned in the finding:

12. user (2026-06-22T09:45:50.917Z)

```
15:BOOT_EPOCH=$(date +%s)
80:UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
84:  python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
```

13. assistant (2026-06-22T09:45:52.878Z)

Let me analyze the bash arithmetic behavior with unset variables:

14. user (2026-06-22T09:45:53.927Z)

```
    12  set -uo pipefail
    13  exec > >(tee -a /var/log/nightly-regression.log) 2>&1
    14  echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
    15  BOOT_EPOCH=$(date +%s)
    16
    17  md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
    18  NAME=$(md instance/name)
    19  ZONE=$(md instance/zone | awk -F/ '{print $NF}')
    20  REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
    21  GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
    22  BUCKET=$(md instance/attributes/gcs-bucket || true)
    23  REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
    24  WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
    25  DO_EMAIL=$(md instance/attributes/email || echo true)
    26  SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
    27  SMTP_USER=$(md instance/attributes/smtp-user || true)
    28  EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
    29  DATE=$(date -u +%F)
    30
    31  # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
    32  cleanup() {
    33    echo "===== self-delete $NAME ($ZONE) ====="
    34    gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
    35      echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
    36  }
    37  trap cleanup EXIT
    38
    39  fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
    40    echo "RUN FAILED ? attempting a failure alert"
    41    if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
    42      cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
    43      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    44      python - <<PY || true
    45  from scripts.nightly_email import send_report
    46  send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
    47              "The nightly engine-regression run failed on the VM before producing a
report.\n"
    48              "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
    49  PY
    50    fi
    51  }
    52  trap 'fail_email' ERR
    53
    54  # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
```

```
    55  curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
    56    sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
    57
    58  # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
    59  mkdir -p ~/rally && cd ~/rally
    60  gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
    61  tar -xzf /tmp/rally.tgz -C ~/rally
    62  echo "$GIT_SHA" > ~/rally/GIT_SHA
    63
    64  # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
    65  sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
    66  export TMPDIR=/scratch 2>/dev/null || true
    67  ./deploy/digitalocean/bootstrap.sh --gpu
    68  . .venv/bin/activate
    69  pip install -e '.[storage-gcs]'
    70  bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
    71
    72  # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
    73  if [ -n "$WEIGHTS_URI" ]; then
    74    mkdir -p ~/models/WASB-SBDT/pretrained_weights
    75    gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
    76    export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
    77  fi
    78
    79  # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
    80  UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
    81  EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
    82  set +e
    83  NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    84    python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
    85  RUN_RC=$?
    86  set -e
    87  echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
    88
    89  # 6) Upload the dated report to GCS for the record (alongside the email).
    90  if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
```

15. assistant (2026-06-22T09:45:57.481Z)

Perfect. Now let me verify the behavior. The critical question is: **what happens at line 80 if `BOOT_EPOCH` is unset?** Let me check the bash option that's set:

At **line 12**, we have `set -uo pipefail`. The `-u` flag means **"treat unset variables as an error"**. This is crucial.

Let me verify by examining what would happen with this configuration:

16. user (2026-06-22T09:45:59.974Z)

```
Testing: BOOT_EPOCH is unset
/tmp/test_unset.sh: line 7: BOOT_EPOCH: unbound variable
```

17. assistant (2026-06-22T09:46:03.504Z)

Excellent. Now let me verify what the claim said?it claimed UPTIME would "silently underflow to zero". But we have `set -u`. Let me check if the finding's error case is even plausible:

18. user (2026-06-22T09:46:07.377Z)

```
date +%s succeeded and returned a Unix timestamp
BOOT_EPOCH set to: 1782121567
```

19. assistant (2026-06-22T09:46:16.652Z)

Now let me check if there's any scenario where `date +%s` could fail and leave BOOT_EPOCH unset despite the `set -u` guard:

20. user (2026-06-22T09:46:20.035Z)

```
date failed, BOOT_EPOCH is now: [UNSET]
BOOT_EPOCH is empty/unset
```

21. assistant (2026-06-22T09:46:23.504Z)

Good! So if `date` fails, BOOT_EPOCH would be unset. But line 12 has `set -u`. Let me trace what would actually happen:

22. user (2026-06-22T09:46:26.326Z)

```
UPTIME=0
```

23. assistant (2026-06-22T09:46:30.236Z)

Interesting! When BOOT_EPOCH is empty (not unset), arithmetic still works and treats it as 0. Now let me test the **exact scenario** in the finding: what if BOOT_EPOCH is completely unset (not set to empty string):

24. user (2026-06-22T09:46:33.648Z)

```
Exit code 1
/tmp/test_exact_scenario.sh: line 13: BOOT_EPOCH: unbound variable
```

25. assistant (2026-06-22T09:46:37.795Z)

Perfect! So with `set -u`, an unset BOOT_EPOCH **DOES** cause an error. The finding's claim says **"bash arithmetic treats it as 0, causing UPTIME to be the full wall time, which is INCORRECT"**. But that's not what happens with `set -u`.

However, let me check: could there be a scenario where the `-u` flag is *temporarily disabled* or where it doesn't apply? Let me look at the actual flow more carefully:

26. user (2026-06-22T09:46:40.431Z)

```
1       #!/usr/bin/env bash
2       # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3       # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4       # an interactive session). Reads its config from instance metadata (set by launch.sh).
5       #
6       # Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
7       #        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8       #        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
9       #
10      # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11      #   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
```

```
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
```

```
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

27. assistant (2026-06-22T09:46:47.406Z)

Now I see line 82: `set +e`. This is AFTER line 80 where UPTIME is calculated. So at line 80, `set -u` is still active. But let me check: is there any code path where `set -u` could be disabled between line 12 and line 80?

Looking at lines 12-80, there is **no `set +u`** anywhere. The only flag change is `set +e` at line 82, which affects **error handling** (`-e`), not **unset variable handling** (`-u`).

KEY FINDING: The claim that "bash arithmetic treats it as 0" is **INCORRECT** when `set -u` is active. With `set -u`, an unset `BOOT_EPOCH` would cause an immediate error at line 80: `BOOT_EPOCH: unbound variable`.

However, let me verify one more scenario: What if `date +%s` actually **succeeds** but returns an empty string or non-numeric value? Let me check:

28. user (2026-06-22T09:46:50.193Z)

```
UPTIME=1782121609 (with empty BOOT_EPOCH)
1782121610 - '' = 1782121610
```

29. assistant (2026-06-22T09:47:04.495Z)

Interesting! An **empty string** is treated as 0 in bash arithmetic. But in the script, `BOOT_EPOCH=$(date +%s)` assigns the timestamp value, not an empty string. The only way

BOOT_EPOCH would be empty is if `date +%s` returned nothing (which would require date to fail).

Let me now check: can `date +%s` fail silently in a startup script? Let me review the context:

1. **Line 15**: `BOOT_EPOCH=$(date +%s)` ? no error handling
2. **Line 12**: `set -uo pipefail` ? unset variables cause errors
3. **Reality**: On a fresh GCP VM, `date` is standard and almost never fails

However, the finding claims the issue is theoretically possible. Let me check if the claim's suggested fix would actually help:

The fix suggests:
```bash
BOOT_EPOCH=$(date +%s) || { echo "FATAL: cannot read system time"; exit 1; }
```

This would catch if `date` fails and exit immediately. But with `set -u` already active, line 80 would error anyway if BOOT_EPOCH were unset.

Let me now verify the Python side: Does the Python script handle UPTIME sensibly?

Looking at `scripts/nightly_reelcount.py` lines 73-90 (the cost_report function):

```python
def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
    """Run cost from billed wall-seconds × machine rate ? USD + INR..."""
    rate_per_sec = float(machine_usd_per_hr) / 3600.0
    usage = {billing.WALL_SECONDS: float(billed_seconds)}
    rates = {billing.WALL_SECONDS: rate_per_sec}
    usd, breakdown = billing.compute_cost_usd(usage, rates)
    return {
        "billed_seconds": round(float(billed_seconds), 1),
        ...
    }
```

The function accepts `billed_seconds` and uses it directly. If `UPTIME` were somehow 0 or incorrect, the billing calculation would be wrong.

However, I need to revisit the actual **possibility** of this defect:

1. **Line 15**: `BOOT_EPOCH=$(date +%s)` ? If `date` fails (rare), BOOT_EPOCH is **unset**
2. **Line 80**: `UPTIME=$(( $(date +%s) - BOOT_EPOCH ))` ? With `set -u`, this would **error immediately**
3. **No silent underflow to zero** ? The script would fail, not silently calculate wrong UPTIME

The claim is **REFUTED** by the `set -u` flag at line 12. The suggested scenario would cause a hard error, not a silent underflow.

HOWEVER, let me check one more edge case: what if `date` is somehow modified or broken in the startup environment itself?

30. user (2026-06-22T09:47:07.413Z)

/tmp/test_subshell.sh: line 8: BOOT_EPOCH: unbound variable

31. assistant (2026-06-22T09:47:09.707Z)

Perfect. The `set -u` protection works even in subshells. Now let me make a final comprehensive test to absolutely confirm:

32. user (2026-06-22T09:47:14.667Z)

```
=== Test 1: Normal execution ===
BOOT_EPOCH=1782121634
UPTIME=0

=== Test 2: Simulating date failure (date returns empty) ===
BOOT_EPOCH is empty string
UPTIME=1782121634 (empty treated as 0)

=== Test 3: If BOOT_EPOCH were truly unset (never assigned) ===
/tmp/final_test.sh: line 26: BOOT_EPOCH: unbound variable
```

33. assistant (2026-06-22T09:47:17.278Z)

Now let me verify one critical detail: is there any scenario where the assignment `BOOT_EPOCH=$(date +%s)` could fail silently and NOT trigger `set -u`? Let me test command substitution failures:

34. user (2026-06-22T09:47:20.338Z)

```
Test:
RESULT=[UNSET]
RESULT2=[fallback]
```

35. assistant (2026-06-22T09:47:22.530Z)

Wait! This is interesting. When `$(false)` runs and returns nothing, the variable is set to an empty string, not left unset! Let me verify this more carefully:

36. user (2026-06-22T09:47:27.311Z)

```
After RESULT=$(false):
  [[ -z $RESULT ]] = true
  [[ -v RESULT ]] = true
  RESULT=[]

Unset variable:
  [[ -v UNSET_VAR ]] = false
  Trying to reference it...
/tmp/test_assignment_unset.sh: line 18: UNSET_VAR: unbound variable
```

37. assistant (2026-06-22T09:47:32.968Z)

Excellent! This clarifies it. The assignment `BOOT_EPOCH=$(date +%s)` will **always succeed** and set BOOT_EPOCH to something (either a timestamp or an empty string). The variable is **never left unset** by a simple assignment like this.

So:
- If `date +%s` returns a number: `BOOT_EPOCH=<timestamp>` ?
- If `date +%s` returns nothing: `BOOT_EPOCH=""` (empty string, set but empty)
- **BOOT_EPOCH is NEVER unset** after the assignment

Therefore, at line 80, `BOOT_EPOCH` would be either:
1. **A valid Unix timestamp** (normal case)
2. **An empty string** (if date failed/returned nothing)

With `set -u`, case 2 would **NOT** error because BOOT_EPOCH is SET (to empty), not unset.

Let me verify case 2 produces wrong results:

38. user (2026-06-22T09:47:39.475Z)

```
BOOT_EPOCH= (empty)
UPTIME=1782121659
Expected: UPTIME should be ~2 seconds of actual work
```

```
Actual: UPTIME is the full wall time from epoch!
```

39. assistant (2026-06-22T09:47:42.470Z)

```
**THIS IS THE BUG!** Not an "unset variable underflow" as claimed, but an **empty string treated
as zero** scenario that results in UPTIME being incorrectly calculated as the full wall time
since epoch.

Let me verify this is actually problematic for the cost calculation:
```

40. user (2026-06-22T09:47:45.752Z)

```
Correct UPTIME: 180s ? Cost: $0.035000
Wrong UPTIME:   1782121659s ? Cost: $346523.66
Error factor:   9900676x more expensive!
```

41. assistant (2026-06-22T09:47:49.198Z)

```
Dramatic! The cost would be wildly incorrect. But now the key question: **Is this scenario
actually plausible in the real script?**

Let me check if there's any code path where `date +%s` could fail:
```

42. user (2026-06-22T09:47:52.117Z)

```
/usr/bin/date
/usr/bin/date: PE32+ executable (console) x86-64 (stripped to external PDB), for MS Windows, 10
sections
1782121672
```

43. assistant (2026-06-22T09:47:54.514Z)

```
Now let me create a comprehensive final analysis. Let me look at the Python side to understand
what the impact would be:
```

44. user (2026-06-22T09:47:56.252Z)

```
def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
    """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).

    ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-
uptime-seconds``;
    otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for
boot/install)."""
    rate_per_sec = float(machine_usd_per_hr) / 3600.0
    usage = {billing.WALL_SECONDS: float(billed_seconds)}
    rates = {billing.WALL_SECONDS: rate_per_sec}
    usd, breakdown = billing.compute_cost_usd(usage, rates)
    return {
        "billed_seconds": round(float(billed_seconds), 1),
        "gpu_seconds": round(float(gpu_seconds), 1),
        "machine_usd_per_hr": float(machine_usd_per_hr),
        "usd": usd,
        "inr": billing.to_inr(usd, fx_inr_per_usd),
        "breakdown": breakdown,
--
        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
        "",
```

45. assistant (2026-06-22T09:47:58.183Z)

Now let me verify the actual data flow and reporting:

46. user (2026-06-22T09:47:59.202Z)

```
        return 2

    billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else total_wall
    cost = cost_report(
        billed_seconds=billed,
```

47. assistant (2026-06-22T09:48:08.086Z)

Perfect. So the flow is:
1. Shell script: `UPTIME=$(( $(date +%s) - BOOT_EPOCH ))`
2. Shell script: `python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME"`
3. Python: `args.vm_uptime_seconds` is used as `billed_seconds`
4. Python: Cost calculated as `billed_seconds * rate_per_sec`
5. Python: Reported in the email/output

Summary of my findings:

The claim about "silent underflow to zero" is **TECHNICALLY INACCURATE** because:
1. With `set -u`, an unset variable would cause an error, not silent underflow
2. However, there IS a real bug scenario

The REAL BUG:
- If `date +%s` fails on line 15, the variable `BOOT_EPOCH` becomes **empty string** (not unset)
- At line 80, empty string is treated as 0 in bash arithmetic
- `UPTIME` becomes `$(date +%s) - 0`, which equals the current Unix timestamp (~1.78 billion seconds)
- This gets passed to Python as an absurdly large `billed_seconds` value
- Cost calculation becomes catastrophically wrong (off by millions of times)

Is this realistic?
- Very unlikely on a GCP VM (date command is always present)
- But the code has no error checking on line 15
- The fix suggested in the finding is good defensive programming

Let me verify one more time that this is NOT protected by `set -e`:

48. user (2026-06-22T09:48:09.197Z)

```
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
```

49. assistant (2026-06-22T09:48:12.909Z)

`set -uo pipefail` contains:
- `-u`: unset variables error ?
- `-o pipefail`: pipe failures are errors ?
- **No `-e`**: command failures don't stop execution ?

So if `date` returns an error code, the script doesn't stop, it just assigns empty string!

Let me create the final verification:

50. user (2026-06-22T09:48:24.442Z)

```
=== Simulating: date command fails silently ===
BOOT_EPOCH=[] (empty string)
Step 1...
```

```
Step 2...
Step 3...
Step 4...
Step 5...
UPTIME=1782121704 (should be ~5 seconds, but is Unix timestamp!)
Would call: python ... --vm-uptime-seconds 1782121704
Calculated cost: $346523.66
(Should be ~$0.01 for 5 seconds, but is $346523.66!)
```

51. assistant (2026-06-22T09:48:26.869Z)

Perfect! I have confirmed the real bug. Now I can provide my structured findings:

52. user (2026-06-22T09:48:42.509Z)

Structured output provided successfully

53. assistant (2026-06-22T09:48:50.364Z)

Summary

I've thoroughly analyzed the claimed defect and verified it is **REAL**, though the finding's description was imprecise about the mechanism.

The Actual Bug:
- **File:** `deploy/gcp/nightly_regression/run_on_vm.sh`, lines 15 and 80
- **Severity:** HIGH
- **Root cause:** Line 12 has `set -uo pipefail` (notably **no `-e`**), so if `date +%s` fails on line 15, the script doesn't exit; instead, `BOOT_EPOCH` becomes an empty string
- **Consequence:** At line 80, the arithmetic `$(date +%s) - ""` treats empty as zero, producing `UPTIME = [current Unix timestamp]` (~1.78 billion seconds), which is then reported as VM cost, inflating the billing calculation by 9+ million times

Key Difference from Claim:
- The finding said "unset variable" would underflow?technically, `set -u` protects against true unset variables
- The real issue: empty *string* (which IS set, so `set -u` doesn't catch it) treated as zero in arithmetic, causing overflow to Unix timestamp, not underflow to zero

Recommended Fix: Add error handling at line 15 and a sanity check before line 84 to catch suspiciously large UPTIME values (anything > 3600 seconds warrants investigation).